

Contents

1	Documentation technique - SAE Application intelligente	4
1.1	Table des matières	4
2	Accueil	4
3	SAE - Application intelligente	4
3.1	Documentation technique	4
3.2	CI et déploiement	5
4	Présentation	5
5	Présentation du code source	5
5.1	Objet	5
5.2	Organisation	5
5.3	Structure principale	5
5.4	Rôle de cette documentation	5
5.5	À retenir	5
6	init	6
7	Module git_commit_release_notes_generator	6
7.1	Rôle	6
7.2	Sous-modules	6
7.3	Objectif fonctionnel	6
8	config	6
9	Configuration	6
9.1	Description	6
9.2	Variables principales	6
9.3	Emplacement et chargement du fichier .env	7
9.4	Points de vigilance	7
9.5	Exemple d'usage	7
10	models	7
11	Modèles de données	7
11.1	Description	7
11.2	Structures principales	7
11.2.1	DiffFile	7
11.2.2	CommitMessage	8
11.3	Rôle dans le flux	8
12	init	8
13	Package ollama_client	8
13.1	Rôle	8
13.2	Sous-modules	8
13.3	Objectif	8
14	base	9
15	Base Ollama client	9
15.1	Description	9

15.2 Rôle attendu	9
15.3 Utilisation	9
16 embedding	9
17 Embedding Ollama	9
17.1 Description	9
17.2 Statut actuel	9
17.3 Rôle fonctionnel attendu	9
18 exceptions	10
19 Exceptions Ollama	10
19.1 Description	10
19.2 Exceptions prévues	10
19.3 Pourquoi les centraliser ?	10
20 llm	10
21 Client LLM	10
21.1 Description	10
21.2 Flux de travail	10
21.3 Points clés	11
21.4 Fonctions exposées	11
21.5 Dépendances	11
22 init	11
23 Service du générateur	11
23.1 Rôle	11
23.2 Sous-modules	11
23.3 Objectif	11
24 core	12
25 Core du service (Point d'entrée du pont Go ↔ Python)	12
25.1 Description	12
25.2 Contrat d'interface (Règle d'or Go / Python)	12
25.3 Les 9 actions gérées (--action <NOM>)	12
25.4 Fonctions clés	13
26 utils	13
27 Utilitaires de service	13
27.1 Description	13
27.2 Rôle attendu	13
27.3 À compléter	14
28 Présentation	14
29 Présentation des tests	14
29.1 Objet	14
29.2 Types de tests présents	14
29.3 Organisation	14
29.4 Rôle de cette documentation	14
29.5 À retenir	14

30 Git wrapper	15
31 Tests du wrapper Git	15
31.1 Description	15
31.2 Ce qui est testé	15
31.3 Cas de test principaux	15
31.3.11. Parsing d'un diff multi-fichiers	15
31.3.22. Sanitisation des secrets	15
31.3.33. Structure du payload JSON	15
31.4 Points de vigilance	15
32 Ollama LLM	16
33 Tests du client Ollama	16
33.1 Description	16
33.2 Ce qui est testé	16
33.3 Stratégie de test	16
33.4 Cas importants	16
33.4.1 génération réussie au premier essai	16
33.4.2 retry puis succès	16
33.4.3 validation stricte du message	16
33.5 Points de vigilance	16
34 Golden diffs	17
35 Fixtures Golden Diff	17
35.1 Description	17
35.2 Structure de données	17
35.3 Rôle fonctionnel	17
35.4 Exemple de scénarios	17
35.5 Points de vigilance	17
36 Initialisation	17
37 Activation des situations de test	17
37.1 Description	18
37.2 Rôle fonctionnel	18
37.3 Comportement	18
37.4 Utilité dans le projet	18
38 Désinitialisation	18
39 Neutralisation des situations de test	18
39.1 Description	18
39.2 Rôle fonctionnel	18
39.3 Objectif	19
39.4 Comportement	19
39.5 Utilité du projet	19
40 Déploiement de la documentation	19
41 Déploiement de la documentation	19
41.1 Objectif	19
41.2 Fichiers concernés	19
41.3 Configuration MkDocs	19
41.4 Workflow de déploiement	19

41.4.1	Déclenchement	20
41.4.2	Marqueur de version de développement	20
41.4.3	Étapes du job build	20
41.4.4	Étapes du job deploy	20
41.5	Résultat attendu	20
41.6	Bonnes pratiques	20
41.7	Commande de vérification locale	20
42	Tests automatiques via GitHub Actions	21
43	Tests automatisés via GitHub Actions	21
43.1	Objectif	21
43.2	Workflows disponibles	21
43.3	Workflow CI	21
43.3.1	Jobs	21
43.4	Vérification locale	21
43.5	Intérêt du workflow CI	22
44	Sprint 00	22
45	Sprint N°1 : Mise en place du projet	22
46	Sprint 01	22
47	Sprint N°2 : Intégration et développement des fonctionnalités	22

1 Documentation technique - SAE Application intelligente

1.1 Table des matières

Contents

2 Accueil

3 SAE - Application intelligente

Documentation du projet d'application intelligente.

Cette documentation présente l'architecture technique, les tests, le suivi de projet et les mécanismes de CI/CD associés.

3.1 Documentation technique

La documentation technique suit la structure du code source et des tests.

- doc_src_files : documentation des fichiers du dossier src
- doc_test_files : documentation des fichiers du dossier test
- monitoring : suivi des sprints et de l'avancement du projet

3.2 CI et déploiement

Le projet met en place deux éléments essentiels de documentation opérationnelle :

- le [déploiement de la documentation](#) : explication du build MkDocs et de la publication sur GitHub Pages
 - les [tests automatiques via GitHub Actions](#) : explication du lancement des tests et de leur organisation dans la CI
-

4 Présentation

5 Présentation du code source

Cette section regroupe la documentation technique du code applicatif du projet.

5.1 Objet

Elle permet de suivre la structure du dossier `src/` et de comprendre le rôle de chaque module, en gardant une correspondance claire entre le code et sa documentation.

5.2 Organisation

Le dossier `src/` contient le cœur applicatif du projet, notamment :

- la génération des messages de commit
- l'intégration avec Ollama
- la gestion Git et des diff
- les composants CLI
- les services de synthèse et d'analyse

5.3 Structure principale

- `git_commit_release_notes_generator/` : logique métier du projet
- `ollama_client/` : intégration avec les modèles LLM
- `service/` : services de traitement, Git et utilitaires
- `cli/` : interface en ligne de commande et vues utilisateur

5.4 Rôle de cette documentation

Chaque fichier de code a un équivalent Markdown dans ce dossier, avec la même organisation arborescente. Cela permet de :

- retrouver rapidement la description d'un module
- comprendre les responsabilités de chaque composant
- maintenir la documentation synchronisée avec l'évolution du code

5.5 À retenir

La documentation du code source doit rester alignée sur l'implémentation réelle. Quand un module est ajouté, renommé ou modifié, la documentation associée doit être mise à jour dans ce dossier.

6 init

7 Module git_commit_release_notes_generator

Fichier source associé : `src/git_commit_release_notes_generator/init.py`

7.1 Rôle

Ce package regroupe le cœur de la génération des messages de commit et des notes de release. Il centralise les données de configuration, les structures de données partagées et les clients externes qui interagissent avec Ollama.

7.2 Sous-modules

- [config.md](#) : paramètres de configuration de l'application.
- [models.md](#) : modèles de données partagés.
- [ollama_client](#) : couche de communication avec Ollama.
- [service](#) : logique métier et traitements applicatifs.

7.3 Objectif fonctionnel

Le package sert de point d'entrée logique pour :

- analyser un diff Git ;
 - filtrer les données sensibles ;
 - transmettre un contexte structuré au modèle LLM ;
 - générer un message de commit conforme à Conventional Commits ;
 - produire éventuellement des éléments exploitables pour des notes de version.
-

8 config

9 Configuration

Fichier source associé : `src/git_commit_release_notes_generator/config.py`

9.1 Description

Ce module contient les paramètres globaux utilisés par le client Ollama et le reste de l'application.

9.2 Variables principales

- `OLLAMA_BASE_URL` : URL du serveur Ollama (ex: `http://10.22.28.190:11434` ou `local`).
- `OLLAMA_MODEL` : modèle utilisé pour la génération des messages de commit (ex: `gemma4:26b`, `gemma4:12b`).
- `OLLAMA_TIMEOUT_S` : délai de requête avant timeout en secondes.

- APP_LANGUAGE : langue de l'application et des messages générés (fr ou en).
- MOCK_INTERFACE : activation du mode simulation/démo hors-ligne (true ou false).

9.3 Emplacement et chargement du fichier .env

- **Emplacement strict** : Le fichier .env réside **exclusivement à la racine du dépôt du projet** (SAE_sujet_8_application_intelligente/.env), ou à l'emplacement indiqué par la variable d'environnement système SMART_COMMIT_APP_ROOT.
- **Aucune pollution** : Le module ne lit ni n'écrit jamais de .env dans src/ ou dans le répertoire de travail courant Path.cwd(), évitant toute confusion ou parasitage d'un dépôt Git tiers analysé par l'outil.
- **Surcharge** : Les variables définies directement dans l'environnement système ont priorité sur le contenu du fichier .env.

9.4 Points de vigilance

- La configuration est lue dynamiquement sans nécessiter de recompilation ni de modification de code.
- Les valeurs par défaut restent compatibles avec l'environnement de l'IUT et la CI.
- Le fichier modèle .env.exemple est versionné à la racine pour guider le déploiement initial.

9.5 Exemple d'usage

Le client LLM et le service core.py importent cette configuration pour construire les requêtes HTTP vers Ollama et configurer la langue de traitement.

10 models

11 Modèles de données

Fichier source associé : src/git_commit_release_notes_generator/models.py

11.1 Description

Ce module définit les structures de données transverses utilisées entre les composants du projet.

11.2 Structures principales

11.2.1 DiffFile

Représente un fichier modifié dans un diff Git.

Champs importants :

- path : chemin du fichier ;
- status : état du fichier (A, M, D, R, etc.) ;
- added / removed : nombre de lignes ajoutées et supprimées ;
- is_binary : indique si le fichier est binaire ;
- patch : contenu brut du diff.

11.2.2 CommitMessage

Représente le message final de commit produit par le modèle.

Champs importants :

- type : type Conventional Commits (feat, fix, docs, chore, etc.) ;
- scope : portée optionnelle ;
- subject : sujet principal ;
- body : description détaillée optionnelle.

11.3 Rôle dans le flux

Les objets de ce module servent de contrat entre :

- le parsing Git ;
- la génération de texte via Ollama ;
- l'assemblage final du message de commit.

Ils évitent les chaînes non structurées et rendent le code plus robuste et testable.

12 init

13 Package ollama_client

Fichier source associé : `src/git_commit_release_notes_generator/ollama_client/init.py`

13.1 Rôle

Ce package encapsule la communication avec Ollama pour la génération de messages de commit.

13.2 Sous-modules

- [base.md](#) : primitives de base et point d'entrée technique.
- [llm.md](#) : logique de génération de message et validation du résultat.
- [embedding.md](#) : gestion des embeddings, si cette fonctionnalité est ajoutée plus tard.
- [exceptions.md](#) : exceptions métier du client.
- [prompts](#) : prompt système utilisé pour imposer la structure du message.

13.3 Objectif

Le package isole les dépendances externes LLM du reste du projet afin de garder la logique métier testable et plus simple à maintenir.

14 base

15 Base Ollama client

Fichier source associé : `src/git_commit_release_notes_generator/ollama_client/base.py`

15.1 Description

Ce module sert de base commune au client Ollama. Il définit les contrats et les éléments de fondation pour les intégrations avec le modèle.

15.2 Rôle attendu

- centraliser les interfaces de communication ;
- préparer les appels HTTP ;
- exposer une structure cohérente pour les autres sous-modules.

15.3 Utilisation

Les modules spécialisés reposent sur cette base pour éviter de dupliquer la logique de transport et de gestion des erreurs.

16 embedding

17 Embedding Ollama

Fichier source associé : `src/git_commit_release_notes_generator/ollama_client/embedding.py`

17.1 Description

Ce module est dédié aux embeddings, c'est-à-dire à la transformation de texte en vecteurs numériques pour les traitements de similarité ou d'indexation.

17.2 Statut actuel

Le fichier est prévu pour évoluer avec les besoins du projet et peut servir à des cas d'usage futurs, notamment la recherche ou la comparaison de diff/commit.

17.3 Rôle fonctionnel attendu

- convertir des chaînes textuelles en représentations vectorielles ;
 - collaborer avec d'autres modules de recherche ou de classification ;
 - rester isolé de la logique métier principale.
-

18 exceptions

19 Exceptions Ollama

Fichier source associé : `src/git_commit_release_notes_generator/ollama_client/exceptions.py`

19.1 Description

Ce module définit les erreurs spécifiques à la couche LLM et aux échanges avec Ollama.

19.2 Exceptions prévues

- `OllamaCallError` : échec de l'appel au serveur.
- `CommitMessageValidationError` : validation du JSON de réponse impossible ou incohérente.

19.3 Pourquoi les centraliser ?

Cela permet de distinguer clairement :

- les erreurs réseau ;
- les erreurs de format de réponse ;
- les erreurs métier liées au contenu généré.

Cette séparation simplifie le diagnostic et le test des cas limites.

20 llm

21 Client LLM

Fichier source associé : `src/git_commit_release_notes_generator/ollama_client/llm.py`

21.1 Description

Ce module gère la génération du message de commit via Ollama. Il construit le résumé du diff, envoie une requête structurée à l'API de chat, valide la réponse JSON et la transforme en objet métier.

21.2 Flux de travail

1. Rejeter immédiatement une liste de fichiers vide (`ValueError`), sans appel réseau.
2. Lire les fichiers différences et les convertir en résumé exploitable.
3. Construire le prompt système et le message utilisateur.
4. Appeler Ollama via une requête JSON structurée, avec une fenêtre de contexte (`num_ctx`) fixée explicitement.
5. Vérifier que la réponse est bien un JSON conforme.
6. En cas d'erreur, relancer une seconde fois avec le message invalidé en contexte.
7. Convertir la réponse validée en objet `CommitMessage`.

21.3 Points clés

- format : le modèle répond avec un schéma JSON strict ;
- validation : le code refuse les messages trop longs, absents ou non conformes ;
- robustesse : un retry est tenté pour corriger une réponse invalide ;
- fenêtre de contexte : `options.num_ctx` est fixé explicitement dans la requête (`OLLAMA_NUM_CTX`, `config.py`) car Ollama utilise sinon une fenêtre par défaut bien plus petite que celle annoncée pour le modèle, ce qui tronquait silencieusement les diffs volumineux ;
- disponibilité : `check_ollama_reachable()` permet de vérifier rapidement (quelques secondes) si le serveur répond, avant de lancer une génération qui pourrait sinon attendre le timeout complet.

21.4 Fonctions exposées

- `generate_commit_message(diff_files)` : fonction principale, décrite ci-dessus.
- `check_ollama_reachable()` : ping rapide du serveur (timeout court), retourne (`True`, `None`) ou (`False`, message d'erreur) sans jamais lever d'exception. Pensée pour être appelée par le CLI avant `generate_commit_message`, pour distinguer un problème réseau (hors réseau de l'IUT) d'une vraie erreur de génération.

21.5 Dépendances

- [config.md](#)
 - [models.md](#)
 - [exceptions.md](#)
 - [commit_message_system.md](#)
-

22 init

23 Service du générateur

Fichier source associé : `src/git_commit_release_notes_generator/service/init.py`

23.1 Rôle

Ce package rassemble la logique métier pure de l'application. Il sert à orchestrer les traitements et à exposer les fonctions qui manipulent les diff et les données de génération.

23.2 Sous-modules

- [core.md](#) : noyau métier du système.
- [utils.md](#) : utilitaires de traitement et aide fonctionnelle.

23.3 Objectif

Cette couche permet de garder le code de génération séparé des détails de communication réseau ou de parsing Git.

24 core

25 Core du service (Point d'entrée du pont Go ↔ Python)

Fichier source associé : `src/git_commit_release_notes_generator/service/core.py`

25.1 Description

Ce module constitue le point d'entrée exécutable sous forme de sous-processus par le front-end Go (`src/cli/bridge/python_client.go`). Il orchestre l'ensemble des fonctionnalités du back-end Python : - interaction et extraction des statuts et diffs Git via `GitWrapper` ; - filtrage des fichiers et masquage des secrets ; - interrogation du LLM via `generate_commit_message` ; - synthèse des release notes au format Markdown ; - diagnostic et joignabilité du serveur Ollama.

25.2 Contrat d'interface (Règle d'or Go / Python)

1. **Sortie JSON stricte sur stdout** : seul un objet JSON unique est imprimé sur la sortie standard.
2. **Logs et diagnostics sur stderr** : toutes les traces, messages d'information et erreurs passent exclusivement par le module logging sur `sys.stderr`.
3. **Résilience sans crash** : toutes les exceptions sont interceptées pour renvoyer un JSON d'erreur propre `{"success": false, "error": "..."}.`

25.3 Les 9 actions gérées (--action <NOM>)

--action	Arguments	Format de sortie JSON	Rôle et source des données
status	--repo	<code>{"is_git_repo", "repo_path", "branch", "staged_count", "unstaged_count"}</code>	Détermine l'état du dépôt Git via <code>GitWrapper</code>
diff	--repo	<code>{"success", "files": [...], "staged_count"}</code>	Analyse et assainit les fichiers indexés (<code>GitWrapper._parse_diff</code>)
stage-all	--repo	<code>{"success", "message"}</code>	Indexe l'ensemble des fichiers modifiés (<code>git add -A</code>)
generate-commit	--repo, --feedback	<code>{"success", "type", "scope", "subject", "body", "raw_formatted"}</code>	Inférence du commit Conventional Commits avec support du feedback
apply-commit	--repo, --message	<code>{"success", "sha", "message"}</code>	Applique le commit sur la branche courante via <code>GitWrapper.commit()</code>
push	--repo	<code>{"success", "message"}</code>	Pousse vers le dépôt distant configuré via <code>git push</code>

--action	Arguments	Format de sortie JSON	Rôle et source des données
release-notes	--repo, --from, --to	{"success", "from_tag", "to_tag", "markdown", "commit_count"}	Synthèse Markdown des commits entre deux révisions (LLM + repli déterministe)
save-config	--base-url, --model, --timeout, --language	{"success", "message"}	Notification de sauvegarde de configuration
ping-ollama	—	{"success", "reachable", "latency_ms", "in-stalled_models"}	Vérification HTTP de joignabilité et des modèles disponibles sur Ollama

25.4 Fonctions clés

- `action_status(repo_path)` : vérifie la validité du dépôt, gère les branches orphelines (detached HEAD) et dénombre les modifications.
- `action_diff(repo_path)` : expose chaque fichier avec son patch, ses compteurs added/removed, le drapeau binary et la détection `has_secrets_masked`.
- `action_generate_commit(repo_path, feedback)` : extrait le diff staged textuel et le transmet au module LLM avec la consigne optionnelle utilisateur.
- `action_release_notes(repo_path, from_tag, to_tag)` : extrait l'historique et génère une synthèse catégorisée (Features, Fixes, Refactoring, Docs) avec repli déterministe en cas de déconnexion d'Ollama.
- `action_ping_ollama()` : teste `/api/version` et `/api/tags` avec mesure précise de la latence réseau en millisecondes.

26 utils

27 Utilitaires de service

Fichier source associé : `src/git_commit_release_notes_generator/service/utils.py`

27.1 Description

Ce module réunit les fonctions utilitaires destinées à simplifier le traitement, la validation et le filtrage des données transitant entre le diff Git et la génération de message.

27.2 Rôle attendu

- mettre en forme les données ;
- normaliser les entrées ;
- factoriser les blocs de logique réutilisables ;
- isoler les opérations auxiliaires du cœur métier.

27.3 À compléter

Le fichier est vide à l'état actuel ; ce document sert de point d'entrée pour documenter les utilitaires ajoutés ultérieurement.

28 Présentation

29 Présentation des tests

Cette section décrit la documentation technique associée aux tests du projet.

29.1 Objet

Le dossier test/ contient les tests unitaires, d'intégration et de situation. Cette documentation permet de comprendre le rôle de chaque test et la stratégie de validation du projet.

29.2 Types de tests présents

- tests unitaires : vérification des fonctions et composants isolés
- tests de wrapper Git : validation de l'interaction avec Git
- tests Ollama LLM : vérification des appels et réponses du modèle
- golden diffs : tests sur des sorties attendues
- tests de situation : scénarios réalistes du projet

29.3 Organisation

La structure des fichiers de documentation suit celle du dossier test/ pour faciliter la lecture et la maintenance.

29.4 Rôle de cette documentation

Chaque test ou groupe de tests possède une page dédiée afin de :

- expliquer le but du test
- détailler le scénario couvert
- documenter les résultats attendus
- faciliter le suivi qualité du projet

29.5 À retenir

Les tests sont une partie essentielle de la fiabilité du projet. Leur documentation doit rester à jour pour refléter les cas de validation réellement mis en place dans le code.

30 Git wrapper

31 Tests du wrapper Git

Fichier source associé : test/test_git_wrapper.py

31.1 Description

Ce module valide le comportement du wrapper Git utilisé pour analyser les diffs et préparer les données envoyées au système de génération de commit.

31.2 Ce qui est testé

- le parsing d'un diff Git en fichiers structurés ;
- la détection des modifications par fichier ;
- la suppression des données sensibles dans les patches ;
- la création d'un payload JSON exploitable par le modèle ou par l'orchestrateur applicatif.

31.3 Cas de test principaux

31.3.1 1. Parsing d'un diff multi-fichiers

Le test vérifie que le diff est découpé correctement en plusieurs objets, avec pour chaque fichier :

- le chemin du fichier ;
- le statut de modification ;
- les lignes ajoutées et supprimées ;
- le patch complet associé.

31.3.2 2. Sanitisation des secrets

Le test simule un contenu contenant une clé ou une valeur sensible et vérifie que le mécanisme de redaction remplace bien les informations sensibles par une valeur de type [REDACTED_SECRET].

31.3.3 3. Structure du payload JSON

Le test s'assure que les objets renvoyés par le wrapper sont convertibles en payloads normalisés, sans informations parasites ou inutiles. Il vérifie notamment la présence des champs attendus comme path, status, patch et is_binary.

31.4 Points de vigilance

- La logique de parsing doit rester tolérante aux variations de format de diff Git.
- La redaction des secrets doit être systématique avant toute transmission hors du contexte local.
- Les payloads doivent rester structurés et légers pour éviter d'encombrer le modèle LLM.

32 Ollama LLM

33 Tests du client Ollama

Fichier source associé : test/test_ollama_llm.py

33.1 Description

Ce module couvre la logique de validation et d'interaction avec le client Ollama. Il vérifie notamment le comportement du code qui transforme un diff en un message de commit conforme à la norme Conventional Commits.

33.2 Ce qui est testé

- la génération d'un message de commit valide dès le premier essai ;
- le mécanisme de retry lorsqu'une réponse est invalide ;
- l'échec après plusieurs réponses invalides ;
- les règles de validation du message (type, scope, subject) ;
- la compatibilité avec les anciennes structures de données binaires.

33.3 Stratégie de test

Les appels réseau réels sont évités par remplacement de `urllib.request.urlopen` avec un faux objet de réponse. Cela permet de simuler le comportement de Ollama sans dépendre d'un service distant.

33.4 Cas importants

33.4.1 génération réussie au premier essai

Le test vérifie qu'un payload bien formé est accepté et transformé en objet métier avec les bons champs : type, scope et subject.

33.4.2 retry puis succès

Lorsque la première réponse est invalide, le système réessaie avec une seconde réponse. Cela confirme que la logique de récupération est bien mise en place.

33.4.3 validation stricte du message

Le test contrôle que les sujets terminés par un point, ainsi que les types inconnus, sont rejetés pour rester conforme à la convention de messages attendue.

33.5 Points de vigilance

- La validation doit rester stricte pour éviter des messages de commit peu lisibles ou incohérents.
- Le mécanisme de retry ne doit pas masquer des erreurs de fond ou des réponses non exploitables.
- La compatibilité avec les objets historiques de type binary doit être conservée pour ne pas casser les données existantes.

34 Golden diffs

35 Fixtures Golden Diff

Fichier source associé : test/fixtures/golden_diffs.py

35.1 Description

Ce module fournit des scénarios de diff déjà préparés, appelés GoldenDiff, pour simuler des changements de développement classiques et tester le bon classement des modifications par type de commit.

35.2 Structure de données

Chaque fixture contient :

- un name de scénario ;
- un expected_type correspondant au type attendu (feat, fix, docs, refactor, test, etc.) ;
- une liste de DiffFile représentant les fichiers modifiés.

35.3 Rôle fonctionnel

Ces fixtures servent de références pour valider la capacité du système à reconnaître la nature d'un changement à partir de son diff, sans dépendre d'un contexte réel de développement.

35.4 Exemple de scénarios

Parmi les cas couverts, on trouve :

- une fonctionnalité ajoutée (feature) ;
- une correction de bug ;
- une documentation mise à jour ;
- un refactoring ;
- l'ajout de tests ;
- une modification de dépendances ou de build.

35.5 Points de vigilance

- Les scénarios doivent rester représentatifs des modifications réelles du projet.
 - Les types attendus servent de repères de validation, sans remplacer la logique métier du modèle.
 - Les fixtures doivent rester stables pour assurer des tests reproductibles.
-

36 Initialisation

37 Activation des situations de test

Fichier source associé : test/situation_test/init_test.py

37.1 Description

Ce script réactive les éléments masqués pour les tests, en restaurant les fichiers et dossiers Git ou sensibles renommés avec le suffixe `_test`.

37.2 Rôle fonctionnel

Il sert à remettre dans leur état nominal les éléments suivants :

- `.git` → `.git`
- `.gitignore` → `.gitignore`
- `.github` → `.github`
- `.gitattributes` → `.gitattributes`
- `.gitmodules` → `.gitmodules`
- `.env` → `.env`

Le suffixe `_test` est utilisé pour neutraliser les éléments sensibles sans les supprimer du dépôt.

37.3 Comportement

Le script parcourt le dossier racine donné, recherche les entrées portant le suffixe `_test`, puis les renomme vers leur nom original. Il traite d'abord les éléments les plus profondément imbriqués pour éviter les conflits de renommage.

37.4 Utilité dans le projet

Cette étape est nécessaire lorsque les situations de test doivent être exécutées dans un environnement proche du vrai dépôt, notamment pour tester les interactions Git ou les fichiers de configuration sensibles.

38 Désinitialisation

39 Neutralisation des situations de test

Fichier source associé : `test/situation_test/deinit_test.py`

39.1 Description

Ce script masque les éléments Git et sensibles du projet en ajoutant le suffixe `_test` aux fichiers et dossiers concernés. Il s'agit du mécanisme inverse du script d'activation.

39.2 Rôle fonctionnel

Il protège l'environnement de travail en neutralisant automatiquement :

- `.git` → `.git_test`
- `.gitignore` → `.gitignore_test`
- `.github` → `.github_test`
- `.gitattributes` → `.gitattributes_test`
- `.gitmodules` → `.gitmodules_test`
- `.env` → `.env_test`

39.3 Objectif

Cette neutralisation empêche les outils externes, la plateforme de dépôt ou les scanners de sécurité de considérer les dossiers de test comme des ressources réelles du projet.

39.4 Comportement

Le script parcourt les sous-dossiers et renomme les éléments ciblés. Il traite les chemins les plus profonds en premier pour limiter les conflits de renommage lorsque plusieurs éléments sont imbriqués.

39.5 Utilité du projet

C'est une étape de sécurité et de reproductibilité essentielle pour les situations de test interactives, notamment dans des scénarios où le dépôt ou des variables d'environnement ne doivent pas être réellement exploités.

40 Déploiement de la documentation

41 Déploiement de la documentation

Cette page explique comment la documentation du projet est construite, publiée et mise à disposition via GitHub Pages.

41.1 Objectif

Le projet utilise MkDocs Material pour générer un site de documentation statique à partir des fichiers Markdown présents dans le dossier docs/. Une fois la documentation validée, elle est déployée automatiquement sur GitHub Pages.

41.2 Fichiers concernés

- mkdocs.yml : configuration du site MkDocs
- .github/workflows/documentation.yml : workflow GitHub Actions de build et déploiement
- docs/ : contenu Markdown de la documentation

41.3 Configuration MkDocs

Le fichier mkdocs.yml définit :

- le nom du site : SAE - Application intelligente
- le thème : material
- la langue : fr
- la navigation du site via la clé nav

Le build MkDocs transforme les fichiers Markdown en un site statique dans le dossier site/.

41.4 Workflow de déploiement

Le workflow GitHub Actions est défini dans .github/workflows/documentation.yml.

41.4.1 Déclenchement

Le build de documentation est exécuté lors de :

- un push sur les branches main et Dev
- un pull_request ciblant main ou Dev
- un déclenchement manuel via workflow_dispatch

Le déploiement officiel sur GitHub Pages reste réservé à la branche main.

41.4.2 Marqueur de version de développement

Quand la documentation est générée depuis une branche autre que main, le titre du site reçoit un suffixe * pour signaler qu'il s'agit d'une version en cours de développement.

- * = documentation en cours de développement
- sans * = documentation officielle de la branche principale

41.4.3 Étapes du job build

1. actions/checkout : récupération du code source
2. actions/setup-python : installation de Python 3.12
3. installation de MkDocs Material
4. ajout du marqueur * pour les branches non principales
5. exécution de mkdocs build
6. publication d'un artefact de prévisualisation pour validation
7. publication sur GitHub Pages uniquement sur main

41.4.4 Étapes du job deploy

Le job deploy dépend du job build et ne se lance que pour main. Il publie le contenu sur l'environnement GitHub Pages via :

- actions/deploy-pages@v4

Cela permet d'exposer la documentation officielle en ligne, selon l'URL définie par le dépôt GitHub Pages.

41.5 Résultat attendu

Après un push ou un pull request sur main ou Dev, GitHub Actions reconstruit la documentation. Les branches hors main affichent un suffixe * pour indiquer qu'il s'agit d'une version de développement. Seule la branche main est déployée publiquement sur GitHub Pages.

41.6 Bonnes pratiques

- garder les fichiers Markdown dans docs/ cohérents avec la structure du projet
- mettre à jour mkdocs.yml si un nouveau chapitre ou une nouvelle page est ajouté
- valider le build localement avec mkdocs build avant de pousser

41.7 Commande de vérification locale

Pour construire le site en local :

mkdocs build

Cette commande crée le dossier `site/`, qui est le même contenu que celui déployé sur GitHub Pages.

42 Tests automatiques via GitHub Actions

43 Tests automatisés via GitHub Actions

Cette page décrit comment les tests du projet sont lancés automatiquement par les workflows GitHub Actions.

43.1 Objectif

Le projet utilise l'intégration continue pour vérifier automatiquement la qualité du code à chaque pull request et pour exécuter les tests importants dans un environnement standardisé.

43.2 Workflows disponibles

Deux workflows principaux sont présents dans le dossier `.github/workflows/` :

- `ci.yml` : exécution des tests automatisés
- `documentation.yml` : build et publication de la documentation

43.3 Workflow CI

Le fichier `.github/workflows/ci.yml` est déclenché sur les pull requests ciblant les branches `main` et `Dev`.

43.3.1 Jobs

43.3.1.1 python-tests Ce job a été retiré car les tests python ont besoin de la connexion à l'IUT pour questionner l'IA local à cet effet, il est donc impossible que le workflow vérifie les tests

43.3.1.2 go-tests Ce job exécute les tests Go du projet :

1. configuration de Go à partir de `src/cli/go.mod`
2. exécution de :

```
go test ./...
```

Il est lancé depuis le dossier `src/cli` pour vérifier le sous-projet Go.

43.4 Vérification locale

Pour lancer les tests rapides localement :

```
python -m pytest -m "not slow" -q
```

Pour lancer les tests lents :

```
python -m pytest -m slow -q
```

Pour lancer les tests Go :

```
cd src/cli  
go test ./...
```

43.5 Intérêt du workflow CI

Le workflow CI sert à :

- détecter les régressions rapidement
 - vérifier les modifications avant fusion
 - garantir une certaine qualité de code sur les branches principales
 - conserver une exécution des tests lourds dans un cadre contrôlé
-

44 Sprint 00

45 Sprint N°1 : Mise en place du projet

User Stories :

- US 1.1 : En tant que développeur, je mets en place un pipeline d'intégration et de déploiement continu (CI/CD)
 - US 1.2 : En tant que développeur, je choisis les différentes technologies à utiliser pour le projet (backend / frontend)
 - US 1.3 : En tant que développeur, je crée des scénarios de test
 - US 1.4 : En tant que développeur, je crée un client pour échanger avec le LLM
-

46 Sprint 01

47 Sprint N°2 : Intégration et développement des fonctionnalités

User Stories :

- US 2.1 : En tant que développeur, je corrige et adapte la récupération des données du git diff afin de les rendre compatibles avec le client LLM.
- US 2.2 : En tant que développeur, je mets en place un client Ollama permettant de générer un message de commit à partir des fichiers modifiés.
- US 2.3 : En tant que développeur, je mets en place la validation des réponses générées par le LLM afin de garantir le respect du format attendu.
- US 2.4 : En tant que développeur, je mets en place une gestion des erreurs et un système de nouvelle tentative lors de la génération du message de commit.
- US 2.5 : En tant que développeur, je définis les contrats DiffFile et CommitMessage permettant d'assurer la communication entre les différents modules.

- US 2.6 : En tant que développeur, je mets en place des tests unitaires et des tests d'intégration pour vérifier le fonctionnement du client LLM.
- US 2.7 : En tant que développeur, je mesure les performances et la conformité des réponses générées par le LLM.
- US 2.8 : En tant que développeur, je mets en place une architecture en miroir du projet afin de faciliter son organisation et sa documentation.
- US 2.9 : En tant que développeur, je réalise un wireframe permettant de définir la structure de l'interface utilisateur.
- US 2.10 : En tant que développeur, je réalise une première version de l'interface utilisateur en Go.

Livrables / DoR & DoD :
